

NetraSetu — Technical Documentation

PS 26038 — Explainable AI for Diabetic Retinopathy Screening in Rural India Team "Game Of Codes": Tanuj (lead, ML), Saad (backend, MATLAB/Simulink), Kankshi, Parth, Vedant

This document explains what the system is, how its parts fit together, and what state each part is actually in today. It reconciles three sources:

- the team's locked design (`docs/system-design-v4.md`);
- the code-verified audit of what in that design is resolved vs. still accurate (`docs/STALE_CLAIMS_AUDIT.md`);
- the final integration-testing pass.

Every ML number here matches `docs/ML_BENCHMARKS.md`, which carries the populations, intervals and caveats. This document explains *why* the system is built the way it is.

1. The problem, and why the architecture looks the way it does

India has 20,944 ophthalmologists, about 15 per million people (AIIMS Delhi national survey, 2025). Blindness is 1.37× more prevalent in rural areas, and 90% of DR-related vision loss is preventable with timely referral.

The real task isn't "build an AI that detects DR." It is: **let a non-specialist technician at a rural PHC do what only a specialist could do before, and let that specialist trust and act on the AI's output in seconds.** Every architectural choice below traces back to that sentence.

Edge/cloud split. Rural PHCs have unreliable internet and modest hardware. Compressing the grading models to run there would sacrifice exactly the accuracy the system is judged on. The only step that genuinely needs to run locally is the image-quality check: the technician needs a retake decision before the patient leaves the chair. So **local does quality gating only; central does all grading.** This applies identically to both front-ends.

Capture source. A phone's bare camera cannot produce a usable fundus photograph; that is an optical limitation, not a software one. Both front-ends therefore take images from a dedicated fundus camera. The mobile app imports those images from the gallery, and it also offers a dedicated fundus-lens capture mode (`mobile_lens` preset). Neither front-end calls an external or unversioned inference endpoint.

Human-in-the-loop, always. The AI never tells a patient they have a disease. Every "probably has DR" outcome is confirmed by an ophthalmologist before it becomes a message to the patient.

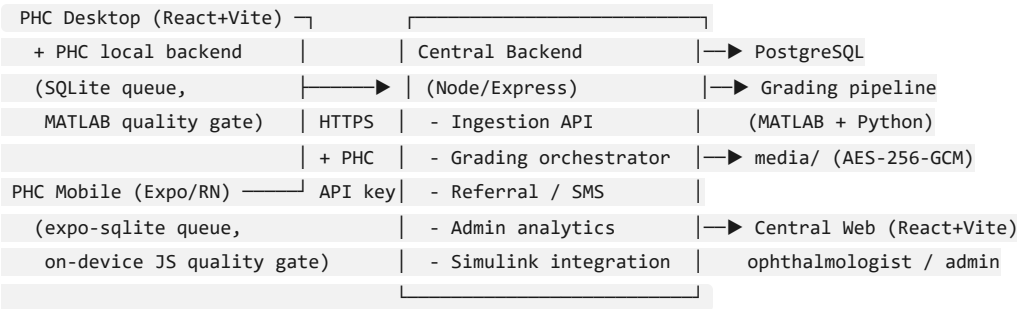
Two front-ends, one spec. The desktop web app and the Expo mobile app implement the same technician workflow against the same central API contract. They differ in exactly one respect: how the image gets onto the device.

No result ever substitutes for a failure. A network failure, a timeout and a working result must be distinguishable on screen. A failed request surfaces as an error or genuinely queues for retry, never as a fabricated result. Mock data only ever appears behind an explicit, visible "DEMO DATA" banner (`DATA_MODE=mock`).

2. Actors

Actor	Touches
PHC technician	Either local front-end (desktop or mobile)
Patient	Gives verbal consent (timestamped at registration); receives an in-person result and/or SMS
Ophthalmologist	Central web app: review queue, case detail, decision controls
District health administrator	Central web app: dashboard, system health, referral tracker, resource recommendations
ASHA / community health worker	Referral follow-up, assigned via the admin interface

3. High-level architecture



Where MATLAB lives:

- the desktop quality gate;
- camera-fingerprint calibration;
- segmentation orchestration;
- the rule engine;
- conformal and calibration math;
- explainability and the evidence report;
- the Simulink resource model.

Branch A (the CNN) and all segmentation models are trained in PyTorch, exported to ONNX and imported into MATLAB's Deep Learning Toolbox. Tensor-level parity was verified for all 9 model artifacts, with maximum differences of 1×10^{-6} to 4×10^{-5} .

Python remains on the serving path for preprocessing and the segmentation worker. The preprocessing chain is implemented exactly once, and that implementation both trains and serves the model: an earlier MATLAB reimplementaion was removed after it flipped real grades via a residual invisible on synthetic test tensors.

No Redis: the grading queue runs in-process in the central backend.

3.1 MATLAB and Simulink in this system

Methodology note. `matlab.codetools.requiredFilesAndProducts` was attempted on each entry-point function first, per the original instruction. It hung indefinitely (10+ minutes, reproduced twice, once with zero other MATLAB processes running, ruling out license contention) on the very first function analyzed — almost certainly because the live MATLAB path's directory is on-path alongside dozens of ONNX-import-generated custom layer classes (`+red_lesion_unet_v2` , `+red_lesion_unet_v1` , etc.), which appears to make MATLAB's static dependency walker effectively intractable here. It was abandoned as impractical in this environment. **Status below**

instead comes from direct code tracing (file:line call chains, confirmed against this session's own live runs and `central.log` output) — a different method than instructed, stated plainly rather than silently substituted.

Component	MATLAB product(s)	Where it runs	Status
Local quality gate	Image Processing Toolbox; MATLAB Compiler (standalone tier)	PHC desktop	Live (<code>quality-gate-matlab/qualityGateMain.m</code> , run via <code>matlab -batch</code> , default engine) for Image Processing. Not built for the MATLAB Compiler tier — <code>quality-gate-matlab/dist/</code> is empty; no <code>.exe</code> has been compiled (\$4.2).
Branch A classifier inference (ONNX import)	Deep Learning Toolbox	Central	Live (<code>gradingOrchestrator.js:187</code> defaults <code>INFERENCE_BACKEND=matlab</code> , no <code>.env</code> override; <code>branchAInferMatlab.m</code> via the persistent MATLAB session).
Segmentation / localization inference	Deep Learning Toolbox	Central	Live for all four models (<code>runMatlabInferenceSession.m</code> : generic <code>predict()</code> on <code>vessel_unet_v1</code> , <code>localization_v1</code> , <code>bright_lesion_unet_v1</code> , and — wired in this same integration pass — <code>red_lesion_unet_v2</code> ; see this doc's §6.6 and the commit "Wire <code>red_lesion_unet_v2</code> into MATLAB serving").
Rule engine (ICDR 4-2-1)	MATLAB (base, no toolbox)	Central	Live (<code>ml-pipeline/grading/ruleEngineGrade.m</code> , called from <code>runCasePipeline.m</code> every graded case; confirmed it both skips quadrant logic and is gated by <code>foveaUnreliable</code>).
Conformal prediction / calibration math	MATLAB (base; Statistics and Machine Learning Toolbox usage not independently confirmed — see methodology note)	Central	Live (<code>ml-pipeline/calibration/conformalTiering.m</code> , feeds the real tier decision in <code>gradingOrchestrator.js</code> ; 90/90 automated tests pass, per <code>ML_BENCHMARKS.md</code> §7).
Grad-CAM + lesion-attention consistency	Deep Learning Toolbox	Central	Live , both halves. <code>lesionAttentionConsistency.m</code> runs inside <code>runCasePipeline.m</code> for every case and is written to <code>explainability_outputs</code> every time, not only in an offline script.
Camera-fingerprint calibration	Image Processing Toolbox	Central	Live (<code>ml-pipeline/cameraCalibration/classifyCameraFamily.m</code>). Its output reaches the tier decision through the camera-probation/mismatch path (<code>gradingOrchestrator.js</code>), not a separate direct channel — still a real, live effect on routing, not dead code.
Evidence-report PDF	MATLAB Report Generator (core-MATLAB fallback renderer)	Central	Live (<code>ml-pipeline/explainability/generateReport.m</code> , generated on demand via <code>services/caseReport.js</code> 's <code>getOrCreateReport()</code> — exercised directly earlier in this integration pass). Whether the live machine actually has MATLAB Report Generator installed (vs. running the fallback renderer via <code>ensureReportGeneratorOnPath.m</code>) was not independently re-checked this pass.
District resource model	Simulink + SimEvents	Central (scheduled)	Built, not on the live path by default. <code>netraSetuPipeline.slx</code> exists and runs to completion when invoked. But (a) the scheduled weekly cross-validation job is currently disabled — confirmed directly from this session's own backend startup log: <code>[simulinkValidation] disabled (SIMULINK_VALIDATION_ENABLED)</code> ; (b) even when enabled, its role is a periodic cross-validation check against a separate function, <code>referenceQueueingModel.m</code> — which is what actually generates the Resource Recommendations screen's live data (<code>services/resourceRecommendations.js</code> 's own header comment: " <i>the .slx stays the PS deliverable and the validation check, not something run per refresh</i> "). See §7 for the corrected description.

4. Local PHC application (both front-ends)

4.1 Screens

Screen	What it does
Patient Registration	New-patient form. Fuzzy duplicate check by name/age/phone before an ID is minted. Timestamped verbal-consent checkbox. Patient symptom/risk questionnaire (diabetes duration, glycemic control, BP, pregnancy, symptoms; tap-only, no skip)
Capture	Desktop: camera trigger / import. Mobile: gallery import or fundus-lens mode. Tags left/right eye before anything else
Quality Result	Pass / borderline / fail with a specific reason (blur, dark, field of view, glare, motion, eyelash occlusion, ...). A real quality score when one exists, "not available" when it doesn't, never a fabricated number. The engine that ran is shown
Capture Metadata Questionnaire	Device, pupil status, lighting, observed issues. Tap-only, under 45 seconds, runs after the quality gate
Local Queue	Captured → Quality-Passed → Synced → Result-Pending → Result-Delivered, with a storage-pressure indicator
Sync Status	Online/offline indicator and pending count. "Synced" means central actually accepted the case (HTTP 201), never merely that a request returned

Final flow on both apps: registration + patient questionnaire → capture → quality gate → capture-metadata questionnaire → queue → sync.

4.2 Local backend

Service	Responsibility
Quality-Gate Engine	Desktop: MATLAB (<code>matlab -batch</code>) is the live path. A compiled-executable tier is coded into the fallback chain (<code>buildQualityGateExe.m</code>) but no .exe has actually been built (<code>quality-gate-matlab/dist/</code> is empty) — it is a designed, not-yet-compiled tier. A pure-JS tier exists and runs only when explicitly enabled (<code>QUALITY_GATE_ALLOW_FALLBACK</code>), and provenance records which engine ran. Mobile: the same JS implementation as its designed primary path (engine reported as <code>js-device</code>), ported once, so both front-ends apply identical rules
Local API	Wraps capture + quality gate and reads/writes the local database
Sync Manager	Prioritizes by urgency tier, then age. Sends a lightweight case-summary packet ahead of the full image on thin connectivity. Uploads full images in resumable chunks with hash verification. Every submission carries the local <code>capture_id</code> as an idempotency key, so a retry never creates a duplicate case

4.3 What differs on mobile, and only this

- **Local persistence:** a real embedded database (`expo-sqlite` , per-record), not a serialized blob.
- **Identity:** the exact same collision-safe ID scheme as desktop (PHC code + timestamp + random suffix).
- **Questionnaires:** both apply in full.
- **Login:** the mobile app authenticates against the paired PHC PC's technician accounts over an encrypted peer channel, with an honest offline-cache fallback.

5. Central system

5.1 Ophthalmologist interface

Screen	Components
Review Queue	Sorted so the least-confident cases surface first, not the most severe. Shows patient ref, eye, PHC, capture time, both branches' grades, tier and claim status. Auto-refreshes
Case Detail	Fundus image with toggleable Grad-CAM overlay. Lesion-evidence panel: microaneurysms, haemorrhages and hard exudates measured, cotton-wool spots shown as unmeasured. Both grading branches side by side with a disagreement flag. Calibrated confidence and conformal tier. Review-duration timer. Engine-provenance details (collapsed by default). Downloadable evidence-report PDF
Decision Controls	Opening a case claims it; a second reviewer sees who holds it and can't submit a conflicting decision. Confirm, or Override with a structured reason, optional free text and a corrected grade. On branch disagreement, Confirm is not available: the reviewer must pick an explicit final grade
Case History	Per-patient, per-eye longitudinal view

5.2 District admin interface

Screen	Components
Dashboard	Cases today/week/total, average review time, override rate, average confidence, cases by PHC
System Health	One consolidated view: silent PHCs (one definition and threshold everywhere), stuck grading jobs, MATLAB session health, referable cases unreviewed past a threshold
PHC Health	Every PHC with last sync time, 24-hour volume, pending/failed count, and active/silent status
Referral Tracker	referred → contacted → attended / lost. Assigned-worker field. Automatic manual-follow-up state when SMS delivery fails
Resource Recommendations	Staffing/routing guidance from the Simulink resource model

5.3 Central backend services

- Ingestion API (dedup on `capture_id`)
- Grading Pipeline Orchestrator (preprocessing → camera calibration → segmentation → dual-branch grading → confidence routing → rationale)
- Referral & Notification Service (Twilio SMS, manual follow-up on failure)
- Admin Analytics Aggregator
- Simulink Integration
- Grading Job Watchdog
- MATLAB Session Supervisor (auto-restart, health reporting)
- Auth Service
- Audit Logging
- Site & Camera Probation Tracking

5.4 Database

PostgreSQL with migration tooling (`node-pg-migrate`). Core tables:

- `patients` , `cases` ;
- `grading_results` : both grades, agreement flag, confidence, uncertainty, conformal tier, claim state;
- `segmentation_outputs` : lesion counts per category, NV score recorded for research only, fovea-reliability flag;

- `explainability_outputs` : Grad-CAM path, report path, engine provenance;
- `ophthalmologist_reviews` , `referrals` , `phc_sites` , `users` , `access_log` .

5.5 API contract

One contract, `docs/api-contracts.md` , consumed identically by both front-ends. Core endpoints:

- `POST /api/v1/cases` (idempotent on `capture_id`);
- `POST /api/v1/cases/summary` ;
- chunked upload endpoints;
- `GET /api/v1/ophthalmologist/queue` ;
- `POST /api/v1/cases/:id/review` ;
- `GET /api/v1/phc/cases/:captureRef/report` ;
- `GET /api/v1/admin/*` (dashboard, system health, PHCs, referrals);
- `POST /api/v1/auth/*` .

6. ML pipeline

6.1 Image quality assessment (local)

Classical computer-vision checks (focus/blur, illumination, contrast, field of view, glare, motion, eyelash occlusion, colour balance, border proportion) with per-camera-family presets. They are implemented once in MATLAB and once as an equivalent JS implementation. The JS version was numerically verified against MATLAB's reference values, to within about 0.005.

6.2 Preprocessing (central)

CLAHE, illumination normalization, denoising and a Ben Graham-style circular crop, implemented exactly once (see §3 for why).

6.3 Camera-fingerprint calibration (central)

Identifies the capture device family from vignetting, aspect ratio and colour-channel gains, cross-checked against the technician-reported device. A mismatch, or an unvalidated camera family, feeds confidence routing as a reason for human review, not as a pixel-level correction.

6.4 Optic disc / fovea localization (central)

A U-Net heatmap regressor with 16 px (disc) / 32 px (fovea) mean error at native resolution on IDRiD. The fovea heatmap occasionally has no real peak, which would silently rotate the quadrant axis the rule engine relies on. A peak-confidence gate (threshold 0.37, raw heatmap units) marks such cases `foveaUnreliable` : the rule engine then skips quadrant-dependent logic, and the case is forced to at least Tier B.

6.5 Vessel segmentation (central)

A U-Net trained on CHASE_DB1 (Dice 0.777 held out), complemented by a Frangi vesselness filter. There is a real cross-dataset gap (Dice 0.619 on DRIVE). The live threshold is a fixed 0.5; the designed per-domain threshold was not implemented.

6.6 Lesion segmentation (central)

- **Red lesions:** the deployed model is **v2**, a 3-class U-Net (background / microaneurysm / haemorrhage) with class-specific minimum-area filters (MA 5 px, haemorrhage 10 px). It reports real, separate counts per quadrant. Dice is 0.599 merged (MA 0.442,

haemorrhage 0.571) on 16 validation images; that is thin evidence, and it does not prove a gain over v1 (0.535).

- **Hard exudates:** Dice 0.583 per image / 0.733 global on IDRiD.
- **Cotton-wool spots:** an explicit scope exclusion (data too sparse), always reported as unmeasured, never as a false zero.
- **Neovascularization:** a suspicion score was built and tested. It **failed validation** (AUC 0.29 IDRiD, 0.38 Messidor-2, both below chance), so it influences no decision.

Standing rule: the lesion size filters and the rule engine's thresholds are one jointly calibrated unit. Changing one without recalibrating the other silently rescales what the thresholds mean.

6.7 DR severity classification — two independent branches (central)

Branch A (CNN). Deployed model: `branchA_v2c` , EfficientNet-B0 at 512×512, with an ordinal-aware loss and class weighting. It was trained on APTOS 2019 + IDRiD, plus an EyePACS subset in the 5-class loss and photometric domain augmentation.

On the 628-image held-out test set:

Metric	Result
QWK	0.884
Referable sensitivity / specificity (argmax)	92.7% / 92.4%
Grade-4 recall	57.4% (31/54)

The live referral decision uses a calibrated threshold on $P(\text{grade} \geq 2)$, set at 0.3873, not the argmax grade. By 50-fold cross-fit ($n = 1,161$), sensitivity is 95.0% and specificity 91.0% in-domain.

Model choice. v2c was chosen over our own promotion rule's pick (v2b) because its auto-clear tier is far more stable on an unseen camera. This deviation is disclosed in `ML_BENCHMARKS.md` §1d.

Branch B (rule engine). Plain, testable code implementing the ICDR "4-2-1" rule on quadrant-mapped lesion counts, auditable line by line against the clinical text. Its frozen thresholds are `RED_FLOOR=3` , `GRADE3_QUAD_MIN=3` and `RULE_MAX_GRADE=3` . It is deliberately capped at grade 3, since no validated neovascularization signal exists. Exact agreement with ground truth is 60.2% on IDRiD's official test split ($n = 103$).

Fusion. Agreement supports the tier. Disagreement unconditionally forces mandatory review with an explicit resolution. Branch B's value is being an independent, auditable second opinion and a disagreement tripwire, not standalone accuracy.

6.8 Confidence routing (central)

One decision per case combines:

- temperature-scaled confidence;
- referable-stratified conformal prediction (v3);
- branch agreement;
- camera validation / probation status;
- quality flags, including `foveaUnreliable` .

Tier	Meaning	Consequence
A — auto-clear	Singleton conformal set, high confidence, branch agreement, validated camera, no quality flag	Skips the ophthalmologist queue
B — AI-assisted review	Narrow conformal interval, or an unvalidated camera	Full rationale report; target review under 30 s
C — full manual review	Wide interval, branch disagreement, CNN grade-4, probation, or quality flag	No shortcuts

In-domain safety (cross-fit, $n = 1,161$): **0% false auto-clears** of referable and of grade ≥ 3 cases, and **zero grade-4 cases auto-cleared** across 1,000 fold assignments.

On an unseen camera (Messidor-2, $n = 872$): the referable false-auto-clear rate rises to 2.3% (upper bound 5.3%), and sensitivity at the shipped threshold is 75.2%. **This is why unvalidated cameras cannot auto-clear.**

Two coded safety clauses are structurally subsumed by the calibrated threshold today, so they are not presented as active safety layers:

- "P(grade 3) + P(grade 4) > 0.5 $\Rightarrow$ referable";
- "referable threshold demotes Tier A."

6.9 Explainability (central)

Grad-CAM on the classifier's final convolutional layer, restricted to the retinal region, with a lesion-attention consistency score that checks the heatmap against the segmentation masks. It is assembled together with the lesion evidence and the rule-engine criteria that fired into one rationale and a downloadable evidence-report PDF.

The Grad-CAM image is the model's own cropped working image, not the original capture, and the UI captions this.

6.10 Symptom + risk questionnaire — queue-ordering hint, not a confidence adjustment (central)

Correction: no confidence-fusion step exists. The questionnaire (diabetes duration, glycemic control, BP, symptoms) does not adjust the classifier's confidence or grade in any way — grep of the whole grading orchestrator and ML pipeline found no such fusion code. What does exist is `calculateUrgencyScore.m`: a synthetic-data-trained, explicitly-documented **queue-ordering hint only**, used to help sequence which cases a reviewer sees first. `runCasePipeline.m` otherwise only passes the questionnaire through for display text on the case report. The questionnaire never overrides or adjusts the image-based grade — that part of the original claim holds; the "adjusts confidence" mechanism itself does not exist.

6.11 Continual learning (central)

Ophthalmologist overrides and their structured reasons are captured in the same transaction as the review. They are exportable with consent enforcement and deduplication (`scripts/exportTrainingSet.js`). Scheduled fine-tuning, the validation gate and promotion are **not implemented this round**: corrections are captured and exportable, and retraining is a manual, future step.

7. Systems layer — Simulink resource model

A discrete-event simulation built in SimEvents.

- **Entities:** patient images.
- **Arrivals:** each PHC's acquisition rate.
- **Network:** a bandwidth-constrained queue across good / poor / very-poor connectivity tiers.
- **Review:** ophthalmologist review as a limited-capacity server (~30 s for Tier B, several minutes for Tier C, with Tier C pre-empting Tier B).
- **Outputs:** queue length over time, average wait and bottleneck location.

What actually feeds the Resource Recommendations screen. The screen's live data comes from `referenceQueueingModel.m` — a separate, pure-MATLAB queueing model, not a per-refresh run of the `.slx` file (`services/resourceRecommendations.js` , its own header comment: *"the .slx stays the PS deliverable and the validation check, not something run per refresh"*). The SimEvents model (`netraSetuPipeline.slx`) is real, runs to completion, and serves as a periodic cross-validation check against `referenceQueueingModel.m` 's output — it does not itself generate what the screen displays.

Bandwidth and timing parameters are modeled assumptions, not measured field data.

8. Resilience and edge-case handling

- **Extended outages:** urgency-then-age sync ordering and PHC last-contact time feeding System Health apply to both front-ends. Summary packets ahead of the full image and a storage-pressure warning are built on **mobile only** (`netrasetu/sync/syncManager.ts` , `netrasetu/lib/storage.ts`) — the desktop app posts the full multipart body directly (with chunked upload for large images), with no summary-packet or storage-pressure code path. A manual export-to-drive fallback is **not built** on either front-end (`phc-local-app/mobile/README.md` : "Not built yet: export-queue-to-drive").
- **Ungradable images:** after a fixed number of failed retakes, the technician can mark a capture "best effort — ungradable." The flag is sent to and stored by central, but central does not yet read it to force Tier C — that routing effect is a stated follow-up, not current behavior.
- **Duplicate patients:** fuzzy matching at registration; the technician confirms or merges.
- **Per-eye capture:** one visit can produce two independently graded cases.
- **Failed patient contact:** an undeliverable SMS flips the referral to manual follow-up.
- **Collision-safe identity:** PHC code + timestamp + random suffix, with `capture_id` as an idempotency key enforced by a uniqueness constraint.
- **Active monitoring:** silent PHC, stuck job, dead MATLAB session and stale referable case are treated as one failure class, surfaced on one screen.
- **MATLAB session supervision:** the session auto-restarts, and health is reported per component (database, queue, MATLAB, Python, segmentation worker). A case that fails while MATLAB is down is currently marked failed and must be resubmitted; automatic re-queue on recovery is not implemented.
- **Concurrent review:** opening a case claims it.
- **Disagreement:** Confirm is never available when the branches disagree.

9. Security (current state)

Verified against the running system in the integration pass:

- **Central:** authentication enabled. Session-based login with bcrypt (a wrong password gets a genuine 401). Role-based access control is enforced server-side on every endpoint, including media: 401 without a session, 403 for the wrong role. The UI routes enforce the same roles.
- **PHC ingestion:** a per-PHC API key is required on every central ingestion call. Keys are provisioned via CLI and never returned by any API.
- **PHC desktop and mobile:** technician accounts with bcrypt, provisioned via CLI.
- **Encryption at rest (central):** AES-256-GCM for stored images, Grad-CAM overlays and report PDFs (`MEDIA_ENCRYPTION_KEY`).
- **Encryption in transit:** TLS is supported by both backends, and provided by the reverse proxy in deployment (§12).
- **Audit logging:** every access to patient data is recorded in `access_log` .

Stated gaps. This is a prototype-stage security floor, not a production compliance claim:

- The PHC-side local stores (desktop SQLite and mobile database/photos) are **not encrypted** this round.
- There are no refresh tokens, no rate limiting and no formal secret rotation.
- No penetration test or security audit has been performed.

10. Honest limitations (stated on purpose)

1. **Decision support, not diagnosis.** This is a screening decision-support system, not an autonomous diagnostic. Every positive is confirmed by an ophthalmologist.

2. **Domain shift.** Sensitivity is 95.0% in-domain but 75.2% on an unseen camera at the shipped threshold. Local recalibration does not close v2c's gap, because ranking quality is the limit, so unvalidated cameras are always routed to human review.
3. **Top-grade resolution.** Exact grade-4 recall is 57.4% in-domain. It is made safe by threshold-based referral and CNN-grade-4 → Tier C routing, not by the recall itself.
4. **MATLAB licensing.** Central inference currently needs a MATLAB license on the serving machine. The fix is packaging with MATLAB Compiler plus the free MATLAB Runtime. This is coded as a fallback tier for the local quality gate (`buildQualityGateExe.m`) but **no executable has actually been compiled yet** (`quality-gate-matlab/dist/` is empty) — the packaging approach is designed and partially built, not delivered, for either the quality gate or central inference (§12).
5. **Thin red-lesion evidence.** Red-lesion v2 was evaluated on 16 images, the same ones it was tuned on.
6. **Neovascularization and uncovered lesion types.** Neovascularization failed validation and is not used. Cotton-wool spots, venous beading and IRMA are not detected.
7. **Rule engine accuracy.** Exact agreement is 60.2% (n = 103).
8. **Vessel threshold.** There is a cross-dataset gap, and the fixed 0.5 threshold is not domain-adaptive.
9. **In-memory grading queue.** Failed-while-MATLAB-down cases are not auto-re-queued.
10. **Partial test coverage.**
 - conformal: 90/90;
 - fovea gate: 12/12;
 - PHC backend: 41;
 - mobile: 28;
 - central backend: no automated suite.
11. **Simulink parameters** are modeled assumptions, not field data.
12. **DME.** Diabetic macular edema cannot be reliably ruled out from colour fundus photographs alone.
13. **No clinical data.** No real-patient or prospective clinical data has been used.
14. **No subgroup analysis.** No subgroup (demographic) analysis has been done: the public datasets don't carry the needed fields.
15. **Quality gate unvalidated against labels.** The quality gate's thresholds are engineered heuristics, not validated against a gradability-labelled dataset.

11. Datasets

Dataset	Size	Used for
APTOS 2019 (Aravind Eye Hospital, India)	3,662 images, 5-class grade	Classifier training and held-out test
IDRiD (Nanded, Maharashtra, India)	516 grading · 81 segmentation · 516 localization	Classifier training and test, lesion and localization models, rule-engine calibration, official test-split evaluation
EyePACS	Curated subset (10k–24k of 35,126, quality-filtered)	v2c classifier training (5-class loss, weight 0.5)
CHASE_DB1	28 images	Vessel segmentation training / held-out evaluation
DRIVE	20 test images	Vessel cross-dataset evaluation only
Messidor-2	1,744 gradable images, 874 patients	External validation only; never used for training, calibration or thresholds

Only public research datasets are used anywhere in this system, and no real patient data. Two of the core training datasets, APTOS and IDRiD, come from Indian populations and clinics, which is directly relevant to the deployment target. The external test, Messidor-2 (France), deliberately comes from a different population and camera.

12. Deployment

12.1 Target production deployment (the plan)

Component	Where it runs	How
PHC desktop app + PHC local backend	On-site at each PHC, on the technician's PC	Installed locally, fully offline-capable. SQLite queue, MATLAB-compiled quality gate on the free MATLAB Runtime (no license needed)
Mobile app	Technician's Android phone	Installed APK. On-device JS quality gate, expo-sqlite queue
Central backend + grading pipeline	A district/state-hosted Linux server or VM	Docker Compose behind a TLS reverse proxy. MATLAB inference packaged with MATLAB Compiler and run on the free MATLAB Runtime, removing the license dependency. Python worker for preprocessing and segmentation. PostgreSQL with nightly backups
Central web (ophthalmologist / admin)	Static hosting	Same-origin with the API (reverse proxy or host rewrites), so the session cookie and image access work without cross-site exceptions

Why this shape. Grading needs the full-capacity models and a single calibrated environment, so it stays central. Offline capture and quality gating are what a rural PHC actually needs locally.

12.2 Current deployment (online submission demo)

⚠ **To be completed after deployment** — fill in each row with what is actually running, and remove this note.

Component	Hosted on	Status / link
Central web (ophthalmologist / admin)	Vercel	link
PHC desktop web (hosted demo station)	Vercel	link
Central backend (API)	host	link to /health
Database	host	—
ML inference (MATLAB + Python)	host / approach	live grading of new uploads: yes / only while ... / no
Mobile app	APK download	link

What a judge can do on the demo: (fill in: log in with demo credentials, browse pre-graded cases, review, see referrals and admin views, submit a new case — graded / queued).

What differs from production: (fill in: e.g., a hosted "demo PHC station" stands in for an on-site PHC; demo data is public-dataset images only; any limit on live grading).